Projeto: Rede Social Simples

Disciplina: Algoritmos, Estruturas de Dados e Programação — Prof. Joselito Junior

Autores:
Gustavo Soares Lima Rios
Pedro Malcon Cabral Silva
Vinícius Fernandes Vieira

1. Introdução

O projeto consiste em uma rede social simplificada, executada em ambiente de terminal e desenvolvida em Python, com persistência de dados em um banco SQLite denominado rede_social.db. O sistema permite cadastrar e remover usuários, criar e desfazer amizades entre dois usuários, listar os amigos de um usuário específico e listar todos os perfis cadastrados, exibindo também o número de amigos de cada um.

O código foi dividido em sete arquivos, cada um com uma responsabilidade bem definida: usuario.py define a classe Usuario; init_db.py cria o banco de dados e as tabelas na primeira execução; database.py concentra toda a comunicação com o SQLite; rede.py atua como camada intermediária entre a interface e o banco; main.py corresponde ao menu exibido ao usuário; e popular_banco.py e teste_sistema.py são arquivos de apoio, responsáveis, respectivamente, por gerar dados de teste e por executar uma bateria de testes automáticos.

2. Estrutura de Dados Escolhida

O banco de dados é composto por duas tabelas, criadas pelo script init_db.py por meio de comandos CREATE TABLE IF NOT EXISTS, o que garante que a estrutura seja recriada apenas quando necessário, sem afetar dados já existentes.

A tabela usuarios armazena os campos id (chave primária com AUTOINCREMENT, gerada automaticamente pelo SQLite), username, nome e idade. O campo username possui a restrição UNIQUE: assim, caso alguém tente cadastrar um username já existente, o próprio SQLite recusa a inserção, sem que o código precise realizar essa verificação manualmente antes.

A tabela amizades contém os campos usuario1 e usuario2, ambos declarados como FOREIGN KEY referenciando o id da tabela usuarios. Cada linha representa um único vínculo de amizade. Como a amizade é tratada como uma relação bidirecional — se A é amigo de B, o inverso também é verdadeiro — o sistema nunca registra duas linhas para o mesmo par: a verificação de existência de um vínculo é sempre realizada nas duas ordens possíveis, (usuario1 = X AND usuario2 = Y) OR (usuario1 = Y AND usuario2 = X).

A classe Usuario, definida em usuario.py, armazena os quatro atributos correspondentes às colunas da tabela (id, username, nome, idade) e implementa o método __str__ para exibição formatada. Sua função é evitar que o restante do código manipule diretamente tuplas retornadas pelo banco, como (1, 'joao123', 'João Silva', 20); em vez disso, database.py converte cada linha em um objeto Usuario, e os demais módulos acessam os dados por meio de atributos nomeados, como usuario.nome e usuario.id.

3. Principais Funções e Lógica de Cada Módulo

O fluxo geral do sistema segue uma arquitetura em camadas: main.py chama os métodos da classe RedeSocial, definida em rede.py, que por sua vez repassa cada chamada para a função correspondente em database.py. Em nenhum momento main.py executa comando SQL diretamente, e rede.py não contém lógica própria além do repasse — sua finalidade é manter a camada de interface separada da camada de acesso ao banco.

database.py

Concentra toda a lógica de persistência. A função adicionar_usuario() realiza o INSERT dentro de um bloco try/except: caso o username já exista, o SQLite lança sqlite3.IntegrityError, capturado pelo except, que retorna False sem interromper a execução do programa.

A função remover_usuario() executa duas operações em sequência obrigatória: primeiro um DELETE em amizades, removendo todas as linhas em que o id do usuário aparece em usuario1 ou usuario2, e somente depois a remoção da linha correspondente em usuarios. Essa ordem é essencial — caso o usuário fosse removido antes, a tabela amizades manteria referências para um id que não existe mais no banco, gerando dados órfãos.

A função criar_amizade() recusa a operação em três situações: quando os dois usernames informados são iguais (não é permitido ser amigo de si mesmo), quando algum dos dois usuários não existe no banco, ou quando a amizade já está cadastrada — verificação feita consultando as duas ordens possíveis antes de qualquer inserção. A função remover_amizade() segue a mesma lógica de busca bidirecional aplicada ao DELETE, e utiliza cursor.rowcount para confirmar se algum registro foi efetivamente removido; caso o valor seja 0, conclui-se que a amizade não existia.

A função listar_amigos() utiliza um JOIN entre usuarios e amizades, relacionando o id do usuário buscado com qualquer linha de amizade que o contenha, em ambos os lados possíveis, retornando diretamente uma lista de objetos Usuario. Já a função contar_amigos() retorna apenas a quantidade de vínculos de um usuário, sendo usada por main.py para exibir o número de amigos junto a cada perfil listado.

rede.py é composto pela classe RedeSocial, com um método para cada operação do sistema, cada um apenas chamando a função equivalente em database.py — não há lógica própria neste arquivo. usuario.py contém somente a classe Usuario, descrita na seção anterior.

main.py corresponde à interface do sistema, permanecendo em um laço while True com uma estrutura match/case — recurso disponível a partir do Python 3.10 — para tratar cada uma das sete opções do menu. A interface foi estilizada com caracteres Unicode de desenho de caixa (╔ ═ ╚ ┌ │ └, entre outros), organizados em duas funções auxiliares: caixa_dupla(), usada no menu principal, e caixa_simples(), usada nas telas de listagem. Para garantir que esses caracteres sejam exibidos corretamente em qualquer terminal — incluindo o cmd.exe do Windows, cuja codificação padrão não é UTF-8 — a saída do programa é reconfigurada explicitamente com sys.stdout.reconfigure(encoding='utf-8') no início da execução. Há também validação do campo idade: caso o usuário digite um valor não numérico, a conversão int() levanta ValueError, e o programa exibe um aviso sem ser interrompido. O arquivo chama criar_banco() no início de main(), de modo que não é necessário executar init_db.py separadamente antes de cada inicialização.

popular_banco.py limpa as tabelas usuarios e amizades antes de inserir os dados de exemplo, e também reinicia o contador de id por meio de DELETE FROM sqlite_sequence — sem essa limpeza, executar o script mais de uma vez resultaria em tentativas de recriar usuários com username já existente, falhando silenciosamente. teste_sistema.py executa uma bateria de testes automáticos sobre um usuário e um amigo de teste: cadastro, bloqueio de username duplicado, criação de amizade, bloqueio de autoamizade, listagem de amigos, remoção de amizade e remoção de usuário. Cada verificação passa pela função checar(), que compara o resultado esperado ao resultado obtido e exibe, por meio dos contadores globais ok e falhas, um resumo final com a quantidade de testes aprovados e reprovados.

4. Dificuldades e Soluções Encontradas

Uma das primeiras dificuldades foi a ausência de tratamento para o erro do SQLite ao tentar cadastrar um username repetido: nas versões iniciais, esse erro (sqlite3.IntegrityError) interrompia a execução do programa. A solução foi encapsular a inserção em um bloco try/except em database.py, de modo que o sistema apenas informasse que o username já estava em uso, em vez de travar.

A maior dificuldade do projeto foi garantir que a amizade não fosse cadastrada duas vezes em ordem invertida. Inicialmente, a verificação de duplicidade considerava apenas a combinação (usuario1, usuario2) em uma ordem fixa, o que permitia cadastrar joão-maria e, em seguida, maria-joão como amizades distintas, fazendo com que o mesmo par aparecesse duplicado na

listagem. A correção consistiu em realizar a busca considerando as duas combinações possíveis antes de qualquer INSERT.

A ordem das operações dentro de remover_usuario() também precisou de ajuste. Na primeira versão, o usuário era removido antes das amizades relacionadas a ele, o que deixava registros em amizades apontando para um id que não existia mais — o sistema não apresentava erro imediato, mas a listagem de perfis passava a exibir dados inconsistentes. A correção consistiu em inverter a ordem: remover primeiro as amizades, depois o usuário.

Executar popular_banco.py mais de uma vez também gerava problemas, pois o script tentava inserir os mesmos usernames novamente, e adicionar_usuario() simplesmente retornava False para cada tentativa, sem qualquer indicação visível de que nada havia sido inserido. Para resolver, foi adicionada uma rotina de limpeza do banco no início do script, removendo os dados anteriores e reiniciando o contador de id antes de inserir os dados de exemplo novamente.

5. Conclusão

O sistema cumpre integralmente o que foi proposto: cadastra e remove usuários, removendo também as amizades vinculadas a eles, cria e desfaz amizades sem duplicar registros, e lista tanto os perfis quanto os amigos de cada usuário, com os dados armazenados em um banco SQLite que permanece disponível mesmo após o fechamento e a reabertura do programa.

A divisão do código em sete arquivos, cada um com uma responsabilidade específica, facilitou a identificação de problemas durante o desenvolvimento: erros relacionados ao banco eram localizados em database.py, enquanto erros de menu ou de validação de entrada eram localizados em main.py. A verificação bidirecional de amizades e a correção na ordem de remoção de usuários foram as soluções que mais contribuíram para a robustez final do sistema.

O sistema não possui login nem senha, uma vez que essa funcionalidade não foi exigida pelo trabalho e aumentaria a complexidade sem necessidade. Também não foi implementada interface gráfica — optou-se por uma interface de terminal, com bordas desenhadas utilizando caracteres Unicode para melhorar a organização visual da exibição. Funcionalidades como autenticação por senha ou troca de mensagens entre usuários ficaram fora do escopo deste trabalho, mas poderiam ser incorporadas sem a necessidade de reescrever o que já existe, já que rede.py e database.py foram projetados de forma independente da camada de interface em main.py.